

Multi-Agent Reinforcement Learning for Multiple Rogue Drone Interception

Panayiota Valianti, Kleanthis Malialis, Panayiotis Kolios, and Georgios Ellinas

Abstract—Unmanned aerial vehicles (UAVs) are increasingly being utilized for a wide variety of applications. However, malicious or illegal UAV (drone) activity poses great challenges for public safety. To address such challenges, this work proposes a framework based on reinforcement learning (RL) in which multiple UAVs cooperatively jam multiple rogue drones in flight in order to safely disable their operation. The main objective is to select mobility and power level control actions for each UAV to best jam the rogue drones, while also accounting for the interference power received by surrounding communication systems. Simulation experiments are conducted to evaluate the performance of the proposed approach, demonstrating its effectiveness and advantages as compared to a centralized solution.

I. INTRODUCTION

Demand for drones has significantly grown over the recent years, thanks to technological advances, the ease of use and the reduced cost. Drones nowadays are broadly employed in various applications, including aerial photography, package delivery, search and rescue, traffic control, and wireless communication support. By 2026, around 1.84 million drones are predicted to be operating in the USA [1], while the global drone market is expected to reach 41.3 billion USD [2]. Drones, nevertheless, create new challenges and threats to security and public safety, primarily when they maneuver over critical infrastructures and public spaces [3], [4].

Over the recent years, various solutions for countering malicious drones have been proposed, mainly relying on detection and interception techniques. However, these are not adequate and more work is required to ensure that drones do not present any safety threats [5]. For example, even though radio-frequency (RF) jamming is the most adopted defense measure for commercial and military anti-drone systems [6], most national regulations prohibit the use of jammers for non-military applications, since they can interfere with existing communication systems.

This work investigates the control of multiple UAV agents for cooperatively jamming multiple rogue drones with the purpose of disrupting their communication links and sensing receivers, while at the same time maintaining minimal wireless interference to nearby communication systems. A

realistic scenario is considered where the trajectories of the rogue drones are unknown to the UAVs and need to be estimated recursively over time using noisy sensor measurements. Further, the UAVs are able to detect the existence of a rogue drone within their limited sensing range based on a probability function with a maximum value less than one. Except for rogue drone measurements, the UAVs additionally receive false-alarm measurements as the sensors are assumed to be imperfect. The objective of the UAVs is to cooperatively track the rogue drones over time, and to maximize the received jamming power at the rogue drones while simultaneously maintaining the UAV-received power below interference levels. It should be noted that, as in this work the decisions on the mobility controls are taken so as to maximize the received jamming power at the rogue drones, tracking is done implicitly through trying to maximize jamming.

A framework based on reinforcement learning, and more specifically Q-learning, is proposed for cooperatively jamming rogue drones. The proposed learning framework employs a learning agent in each UAV and is scalable with respect to the number of rogue drones and UAVs, while being able to be executed in real-time, thus justifying the utilization of RL. The main contributions of this work are:

- An RL-based framework is proposed in which multiple autonomous UAVs cooperatively jam rogue drones. This framework takes into consideration all the important characteristics of the scenario, including the UAVs' limited sensing range and detection uncertainty, the existence of false alarms, and the unknown rogue drone trajectories.
- The multi-agent/multi-target jamming problem is formulated as a distributed learning problem, where each UAV is an independent learning agent where cooperation is achieved via a novel reward function and minimal communication between the agents. Specifically, each agent learns to maximize the received power at the rogue drones, while keeping the interference power received at the UAVs below a critical threshold.
- The performance of the proposed approach is evaluated through extensive simulation results, demonstrating its feasibility and advantages.

The rest of the paper is structured as follows. Section II summarizes the related work, and briefly introduces RL. Section III presents the system model and formulates the cooperative jamming problem. Section IV describes the proposed RL-based multiple UAV control approach for cooper-

The authors are with the Department of Electrical and Computer Engineering and the KIOS Research and Innovation Center of Excellence (KIOS CoE), University of Cyprus, Nicosia, 1678, Cyprus. E-mail: {pvalia01, kmalia01, pkolios, gellinas}@ucy.ac.cy.

This work was supported by the European Union's Horizon 2020 research and innovation programme under grant agreement No 739551 (KIOS CoE - TEAMING) and from the Republic of Cyprus through the Deputy Ministry of Research, Innovation and Digital Policy.

ative jamming, including a discussion on the computational aspects of the proposed method, while Section V provides detailed simulation results. Finally, Section VI concludes the work.

II. BACKGROUND AND RELATED WORK

Numerous surveys have studied the state-of-the-art counter-drone systems. The authors in [7] describe various technologies for detection and identification of drones, including radar, RF, acoustic, optical, and thermal, and summarizes neutralization technologies, such as jamming, spoofing, hijacking, and capturing. The recent survey in [8] categorizes the counter-drone systems into ground- and sky-based solutions, while the works in [6], [9] review commercially available anti-drone systems. Further, the work in [4] examines the challenges of defending malicious drones near airport facilities and analyzes the advantages and limitations of the existing countermeasures. More details on drone detection, tracking, and mitigation techniques are found in [10]–[12].

Moreover, the authors in [9], [13], [14] develop a terrestrial anti-drone system that is capable of detecting and RF jamming a rogue drone. Once a rogue drone is detected, a high-power jamming signal is directed towards it, forcing it to disconnect from its controller and enter its failsafe mode. For the drone detection mechanism, the approach in [13] uses a radar, while [14] uses RF analysis, and [9] fuses audio signals, computer vision, and RF signals. Further, the work in [15] proposes and implements an aerial counter-drone system in which a commercial drone is equipped with antennas to autonomously detect, track, and jam a rogue drone.

More recent works investigate the use of a swarm of cooperative UAVs to counter the operation of rogue drones. This approach offers many benefits, including efficiency, flexibility of deployment, scalability, and coverage expansion, as described in [16]. The work in [17] proposes a detection-and-tracking system that utilizes a dynamic radar network composed of UAVs outfitted with radars; however, no defense mechanism is considered. Further, in [18], the cooperative UAVs carry a net in order to catch a rogue drone in the air; nevertheless, the presence of a reliable target detection system is assumed. Moreover, the work in [19] proposes an approach where multiple networked UAVs intercept a rogue drone by creating a formation around it so as to restrict its movements and force it to exit a critical area. The assumption is that the rogue drone avoids collisions with the rest of the UAVs, and that a highly accurate drone detection-and-tracking system is in place. Finally, the works in [20], [21] address the problem of tracking-and-jamming rogue drones by multiple cooperative UAVs in 2D space, and propose centralized and distributed decision and control algorithms for tackling this problem. In particular, in [20] a centralized approach is proposed where the team of UAVs jointly decides upon jamming power level and mobility control actions so that the jamming power received at the rogue drone is maximized, while in [21] a distributed control approach is proposed for solving the same problem.

Recently, RL has also been applied in a wide range of UAV-based problems, such as drone racing [22], UAV-aided cellular networks [23], and UAV coverage path planning [24]. Further, in [25], a UAV learns a maneuver for pursuing an evader drone, while in [26] a terrestrial drone defense system is proposed that is equipped with various defense mechanisms and uses Q-learning to learn an optimal defense policy. However, no work on RL-based jamming of multiple rogue drones utilizing a team of UAVs has been reported in the state of the art.

This is precisely the problem addressed in this work. Specifically, an RL-based framework is proposed that uses Q-learning in a distributed manner. In particular, each UAV is equipped with an autonomous agent that maximizes the received jamming power at the rogue drones, while ensuring that the interference power between the UAVs remains below a predefined threshold. In addition, the UAVs apply stochastic filtering to estimate the rogue drones' states.

A. Background on Reinforcement Learning

Reinforcement learning [27] is a paradigm in which an agent interacts with its environment so as to maximize a numerical feedback known as the reward. This feedback is collected in a trial-and-error manner, where the agent tries out different actions and receives responses from the environment, i.e., rewards, relative to its actions. The objective of an RL problem is to find an optimal policy, i.e., to map states to actions, so as to maximize the accumulated reward.

Q-learning, is a popular algorithm in which an optimal policy is learnt by estimating the values of state-action pairs, $Q(s, a)$, and storing them in a table, known as the Q-table. The Q-learning update is given by:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (1)$$

where α is the learning rate and r is the reward received for transitioning from state s to state s' by taking action a .

As many RL applications are subject to large or continuous state and action spaces, they usually require function approximation. Tile coding [27] is a commonly used linear state function approximator that exhaustively partitions the state space into a set of overlapping tilings and tiles.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

1) *Target Dynamics*: In this work, rogue drones (i.e., targets) belonging to set $M = \{1, 2, \dots, |M|\}$ maneuver in the environment with the following discrete-time dynamics:

$$x_k = \zeta(x_{k-1}) + G\nu_k, \quad (2)$$

where $x_k = [x, \dot{x}, y, \dot{y}, \omega]_k^\top \in \mathbb{R}^5$ is the target's state vector at time-step k containing the position (x, y) and velocity $(\dot{x}, \dot{y})$ in 2D Cartesian coordinates, as well as turn rate ω . The process noise $\nu_k \sim \mathcal{N}(0, \Sigma_\nu)$ models the random disturbances in the evolution of the state and follows a normal distribution with zero mean and covariance matrix

$\Sigma_\nu = \text{diag}[\sigma_x^2, \sigma_y^2, \sigma_\omega^2]$. The non-linear function $\zeta(x)$ and the G matrix are given by the coordinated turn model [28], [29]:

$$\zeta(x) = \begin{bmatrix} x + \frac{\dot{x}}{\omega} \sin(\omega \Delta T) - \frac{\dot{y}}{\omega} (1 - \cos(\omega \Delta T)) \\ \dot{x} \cos(\omega \Delta T) - \dot{y} \sin(\omega \Delta T) \\ y + \frac{\dot{y}}{\omega} (1 - \cos(\omega \Delta T)) + \frac{\dot{x}}{\omega} \sin(\omega \Delta T) \\ \dot{x} \sin(\omega \Delta T) + \dot{y} \cos(\omega \Delta T) \\ \omega \end{bmatrix},$$

$$G = \begin{bmatrix} \frac{\Delta T^2}{2} & 0 & 0 \\ \Delta T & 0 & 0 \\ 0 & \frac{\Delta T^2}{2} & 0 \\ 0 & \Delta T & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

where ΔT is the sampling interval. The target dynamics are assumed to satisfy the Markov property as shown in Eq. (2).

2) *Agent Dynamics*: A set of controllable UAV agents is assumed, $S = \{1, 2, \dots, |S|\}$, for counteracting the operation of the rogue drones. At time-step k , the state vector of agent j is denoted by $\bar{s}_k^j = [\bar{s}_x^j, \bar{s}_y^j]^T \in \mathbb{R}^2$, i.e., the position in 2D Cartesian coordinates, and its dynamics are given by:

$$\bar{s}_k^j = \bar{s}_{k-1}^j + a_{l_1, l_2} = \bar{s}_{k-1}^j + \begin{bmatrix} \Delta_R[l_1] \cos(l_2 \Delta \theta) \\ \Delta_R[l_1] \sin(l_2 \Delta \theta) \end{bmatrix}, \quad \begin{matrix} l_1 = 1, \dots, |\Delta_R| \\ l_2 = 0, \dots, N_\theta \end{matrix} \quad (3)$$

where Δ_R is the vector of possible radial step sizes (with $\Delta_R[l_1]$ returning the value at index l_1), $\Delta \theta = 2\pi/N_\theta$, and the parameters $(|\Delta_R|, N_\theta)$ determine the number of possible mobility control actions. The mobility control action a_{l_1, l_2} represents the direction and step size of the movement along the x and y axis. At each time-step k , each UAV agent chooses one mobility control action from the discrete set $\mathbb{U} = \{a_{l_1, l_2} \mid l_1 = 1, \dots, |\Delta_R|, l_2 = 0, \dots, N_\theta\}$, as computed by Eq. (3), while agent j 's set of all admissible mobility controls is denoted by $\mathbb{U}_k^j = \{\bar{s}_k^{j,1}, \bar{s}_k^{j,2}, \dots, \bar{s}_k^{j,|\mathbb{U}_k^j|}\}$.

3) *Agent Sensing Model*: The UAV agents detect nearby targets with a limited sensing range, modeled by function $p_D(x_k, \bar{s}_k)$ which gives the probability of whether a target at state x_k and position Hx_k (where H is a matrix that extracts the (x, y) coordinates from a target's state vector) can be detected by an agent at state $\bar{s}_k$ at time-step k . This probability is modeled according to:

$$p_D(x_k, \bar{s}_k) = \begin{cases} p_D^{\max} & \text{if } d_k < R_0 \\ \max\{0, p_D^{\max} - \eta(d_k - R_0)\} & \text{if } d_k \geq R_0 \end{cases} \quad (4)$$

with $d_k = \|Hx_k - \bar{s}_k\|_2$ denoting the Euclidean distance between the agent and the target at time-step k , $p_D^{\max}$ denoting the detection probability for a target that is positioned within distance R_0 from the agent, and parameter η indicating the reduced effectiveness of the agent to detect distant targets.

Each agent is equipped with a 2D range-finding sensor which receives noisy heading and range measurements $y_k \in \mathbb{R}^2$ from the detected target with state x_k according to the measurement model given by:

$$y_k = h(x_k, \bar{s}_k) + w_k, \quad (5)$$

where $h(x_k, \bar{s}_k) = \left[\arctan\left(\frac{\bar{s}_y - y}{\bar{s}_x - x}\right), \|\bar{s}_k - Hx_k\|_2 \right]^T$ and the measurement noise $w_k \sim \mathcal{N}(0, \Sigma_w)$ follows a zero mean normal distribution with covariance matrix $\Sigma_w =$

$\text{diag}[\sigma_\phi^2, \sigma_\zeta^2]$. The standard deviations $(\sigma_\phi, \sigma_\zeta)$ are range dependent, and given by $\sigma_\zeta = \zeta_0 + \beta_\zeta \|\bar{s}_k - Hx_k\|_2^2$ and $\sigma_\phi = \phi_0 + \beta_\phi \|\bar{s}_k - Hx_k\|_2$.

The agent's 2D range-finding sensor consists of $|M|$ sensing slots for detecting each one of the $|M|$ targets. As sensors are imperfect, the agents additionally receive false-alarm measurements $c_k^1, \dots, c_k^n$ with an average Poisson rate λ at each sensing slot of time-step k . These measurements follow the uniform distribution over the measurement space and are distributed based on the density function $p_c(c)$.

Overall, each agent $j \in S$, at each time-step k and sensing slot i , receives the measurement set Υ_k^{ji} as follows:

$$\Upsilon_k^{ji} = \bigcup \left\{ a \in \{\emptyset, y_k^{ji}\}, b \subseteq \{c_k^1, \dots, c_k^n\} \right\}, \forall i \in [1, \dots, |M|] \quad (6)$$

where y_k^{ji} denotes the measurement received by agent j at sensing slot i , i.e., received from target i .

4) *Agent Power Transmission Model*: Agents are outfitted with omnidirectional antennas for jamming the rogue drones. At each time-step k , agent j 's transmitted power P_k^j varies over a discrete set of power levels $\mathbb{L}$. The received power at a target with state x_k from an agent with state $\bar{s}_k$ and transmit power P_k is subject to the free-space path-loss (FSPL) model [11], [13]:

$$FSPL(x_k, \bar{s}_k) = 20 \log_{10}(\|\bar{s}_k - Hx_k\|_2) + 20 \log_{10}(f) + 32.45 \quad (7)$$

where $\|\bar{s}_k - Hx_k\|_2$ is the Euclidean distance (in meters) between the pursuer agent and the target, and f denotes the carrier frequency (in GHz). The received power at the target is then computed as:

$$P_k / (10^{FSPL(x_k, \bar{s}_k)/10}) = P_k \Lambda(x_k, \bar{s}_k). \quad (8)$$

B. Problem Formulation

The joint mobility and power control decision problem for cooperatively jamming multiple targets can be formulated as:

$$(P1) \max_{P_k^j, u_k^j} \min_{i \in M} \sum_{j=1}^{|S|} P_k^j \Lambda(\tilde{x}_k^{ji}, u_k^j) \quad (9)$$

$$\text{s.t.} \quad \sum_{j' \in S, j' \neq j} P_k^{j'} \Lambda(u_k^j, u_k^{j'}) \leq \Delta, \forall j \in S \quad (10)$$

$$P_k^j \in \mathbb{L}, u_k^j \in \mathbb{U}_k^j, \forall j \in S \quad (11)$$

where $\tilde{x}_k^{ji}$ is the state of target i as predicted by agent j , as the actual target state x_k^i is not available to the agents. Since the agents transmit omnidirectional jamming signals, the constraint in Eq. (10) ensures their safe operation by keeping the received power below interference levels, Δ . The objective function in Eq. (9) finds the mobility controls u_k^j and transmission power levels P_k^j for each agent j to maximize jamming and subsequently tracking of the targets. More specifically, the received jamming power at each target is maximized, by maximizing the minimum target-received power, to encourage similar distribution of jamming power amongst the targets. Assuming agent j transmits power P_k^j to jam target i , the target-received power is maximized as the agent selects mobility controls u_k^j that are closer in distance to the target, since the received power is subject to the path-loss model, described in Eq. (7). Note also that an agent

receives a target measurement only when the target is within the agent's sensing range and is detected with a probability given by Eq. (4). Then, selecting the mobility controls u_k^j for each agent j that maximize the probability of detecting the targets is a sufficient condition for tracking and estimating the target states, which in our case is achieved implicitly, since both jamming and tracking objectives seek to minimize the distance between an agent and a target.

IV. RL-BASED COOPERATIVE JAMMING

This section presents the RL-based approach for jamming and subsequently tracking rogue drones using multiple UAV agents. First, the proposed Q-learning-based framework is described, and then details are provided for target state estimation using stochastic filtering.

A. Proposed Cooperative Q-learning Algorithm

Each UAV agent is an independent learner, having its own Q-table. In order to solve the described cooperative jamming problem using RL, it is formulated as a Markov Decision Process (MDP) [27]:

State: The state vector of agent j at time-step k is determined by the positions of the agents and the predicted position of each target i (which is extracted from the predicted target state $\tilde{x}_k^{ji}$), and is denoted by $s_k^j = [m^{j1}, \dots, m^{j4}, p^{j1}, \dots, p^{j4}, n^{j1}, \dots, n^{j4}, q^{j1}, \dots, q^{j4}]_k$. The state vector of the j -th agent comprises 4 components:

- m^{jl} : number of targets in the l -th area of agent j
- p^{jl} : average Euclidean distance between agent j and the targets in the l -th area of agent j
- n^{jl} : number of agents in the l -th area of agent j
- q^{jl} : average Euclidean distance between agent j and the remaining agents in the l -th area of agent j

where $l \in \{1, 2, \dots, 4\}$. Essentially, at each time-step k each agent j splits the 2D space in four areas by assuming Cartesian axes centered at its current position coordinates and computes relative distances from the agents and targets. Importantly, the size of the state vector is constant and independent of the number of agents and targets and thus allows the framework to scale-well with respect to the number of learning agents and targets, and the state vector is also independent of the environment's area size.

Action: At each time-step k , each agent j selects an action a_k^j from the discrete action space $\mathcal{A} = \mathbb{U} \times \mathbb{L}$, which includes choosing a mobility control and a transmit power level as defined in Sections III-A2 and III-A4, respectively. Illegal actions are restricted, e.g. an action that moves an agent outside of the environment's area.

Reward: After observing its state and taking an action, each agent j receives a scalar reward from the environment. The agents aim to maximize the received jamming power at the targets while satisfying the interference constraints as defined in (P1). The reward at time-step k is defined as follows:

$$r_k^j = \begin{cases} \min_{i \in M} \sum_{j'=1}^{|S|} P_k^{j'} \Lambda(\tilde{x}_k^{ji}, u_k^{j'}) & \text{if (10) is satisfied} \\ -\max_{j'' \in S} \sum_{j'=1}^{|S|} P_k^{j'} \Lambda(u_k^{j''}, u_k^{j'}) & \text{otherwise.} \end{cases} \quad (12)$$

Overall, if the constraint (10) is satisfied, the reward function is defined to maximize the minimum target-received power. However, if the constraint (10) is not satisfied, then at least one agent is interfered with, which results in a penalty equal to the maximum agent-received power.

The RL-based training algorithm runs in a distributed and independent manner at each UAV agent and is summarized in Alg. 1. First, each agent initializes its Q-table with zero values (line 1). For every episode, which lasts for K consecutive time-steps, the environment is initialized randomly, encouraging the agents to learn various strategies via trial-and-error (lines 3). Each agent selects an action according to an ϵ -greedy policy, i.e., selects an action that corresponds to the maximum Q-value in the current state with probability $1 - \epsilon$ or to a random action with probability ϵ (line 6). The exploration parameter ϵ and learning rate α decay over time in order to encourage learning and exploration at initial episodes and exploitation afterwards (line 12). Each agent executes its action, receives the reward, and observes the new state before updating the Q-value of the current state-action pair (lines 7-9).

Algorithm 1 Q-learning training for multi-UAV jamming of rogue drones.

```

1: Initialize Q-tables for each agent with zero values
2: for episode  $n = 1, 2, \dots, N$  do
3:   Initialize environment
4:   for timestep  $k = 1, 2, \dots, K$  do
5:     for agent  $j = 1, 2, \dots, |S|$  do
6:       Choose a greedy action  $a_k^j = \operatorname{argmax}_{a_k \in \mathcal{A}} Q^j(s_k^j, a_k)$ 
         with probability  $1 - \epsilon$  or a random action with proba-
         bility  $\epsilon$ 
7:       Take action  $a_k^j$ 
8:       Observe reward  $r_k^j$  and next state  $s_{k+1}^j$ 
9:       Update  $Q^j(s_k^j, a_k^j)$  using Eq. (1)
10:    end for
11:  end for
12:  Decay  $\alpha$  and  $\epsilon$ 
13: end for
```

B. Rogue Drone State Estimation

The UAV agents use stochastic filtering recursively over time to estimate the target states given noisy and false-alarm measurements. The main focus of stochastic filtering [30] is to compute the conditional posterior distribution $p(x_k | y_{1:k})$ of a target state x_k at time-step k given measurements $y_{1:k} = y_1, y_2, \dots, y_k$ until time-step k . The prediction step and update step are given by the Bayes recursion:

$$p(x_k | y_{1:k-1}) = \int p(x_k | x_{k-1}) p(x_{k-1} | y_{1:k-1}) dx_{k-1} \quad (13)$$

$$p(x_k | y_{1:k}) = \frac{p(y_k | x_k) p(x_k | y_{1:k-1})}{\int p(y_k | x_k) p(x_k | y_{1:k-1}) dx_k}, \quad (14)$$

where $p(x_k | x_{k-1})$ is the transitional density derived from Eq. (2), and $p(y_k | x_k)$ is the likelihood function.

In this work, each UAV agent j keeps a separate density $p_k^{ji}(\cdot)$ for each target i , i.e., the number of targets is fixed and known, and the measurements' assignments to these targets

are also known. As discussed in Section III-A3, at each time-step k , each agent receives a measurement set, so in our case y_k becomes Υ_k^{ji} . At the prediction stage, each agent j computes the predicted target state for each target i , using the prediction density $p(x_k|y_{1:k-1})$ [obtained by Eq. (13)], as follows:

$$\tilde{x}_k^{ji} = \int x_k p^{ji}(x_k|\Upsilon_{1:k-1}) dx_k. \quad (15)$$

The posterior distribution $p(x_k|y_{1:k})$ depends on the measurement likelihood function $p(y_k|x_k)$ which in our case becomes $p(\Upsilon_k^{ji}|x_k^i, \bar{s}_k^j)$. As the UAV agents receive both noisy target and false-alarm measurements (discussed in Section III-A3), the measurement likelihood function of the j -th agent and i -th target is given by:

$$p(\Upsilon_k^{ji}|x_k^i, \bar{s}_k^j) = \left[1 - p_D(x_k^i, \bar{s}_k^j)\right] e^{-\lambda} \prod_{\ell \in \Upsilon_k^{ji}} \lambda p_c(\ell) + e^{-\lambda} p_D(x_k^i, \bar{s}_k^j) \sum_{\ell \in \Upsilon_k^{ji}} p(\ell|x_k^i, \bar{s}_k^j) \prod_{\substack{\varepsilon \in \Upsilon_k^{ji} \\ \varepsilon \neq \ell}} \lambda p_c(\varepsilon) \quad (16)$$

where $p(y_k^{ji}|x_k^i, \bar{s}_k^j) = \mathcal{N}(y_k^{ji}; h(x_k^i, \bar{s}_k^j), \Sigma_\omega)$ is target i 's and agent j 's measurement likelihood function due to Eq. (5). To derive Eq. (16), the fact that $\Upsilon_k^{ji} = \{c_k^1, \dots, c_k^{|\Upsilon_k^{ji}|}\}$ is first used if the agent receives only false-alarm measurements (i.e., it does not receive any measurements from the target) and then $p(\Upsilon_k^{ji}|x_k^i, \bar{s}_k^j)$, is calculated as:

$$|\Upsilon_k^{ji}|! \left[1 - p_D(x_k^i, \bar{s}_k^j)\right] \text{Pois}(|\Upsilon_k^{ji}|) \prod_{\ell \in \Upsilon_k^{ji}} p_c(\ell), \quad (17)$$

where $\text{Pois}(|\Upsilon_k^{ji}|) = \frac{\lambda^{|\Upsilon_k^{ji}|} e^{-\lambda}}{|\Upsilon_k^{ji}|!}$ is the Poisson distribution with rate parameter λ which gives the probability of obtaining exactly $|\Upsilon_k^{ji}|$ false-alarm measurements at time-step k and $\prod_{\ell \in \Upsilon_k^{ji}} p_c(\ell)$ is the joint density of the false-alarms, assuming they are independent. By adjusting this expression to consider all possible permutations of the measurement sequence and the fact that the target has not been detected, the first term of Eq. (16) is derived. The second term of Eq. (16) accounts for the case where agent j has detected target i , meaning that apart from false-alarm measurements, agent j has also received measurement y_k^{ji} .

At the update stage, i.e., after receiving measurement sets Υ_k^{ji} for time-step k , the posterior distribution is computed by Eq. (14) and the estimated target state $\hat{x}_k^{ji}$ is derived as:

$$\hat{x}_k^{ji} = \int x_k p^{ji}(x_k|\Upsilon_{1:k}) dx_k. \quad (18)$$

The agents then exchange their local estimation results, in order to fuse them using covariance intersection [31] and obtain the final estimate for the target state $\hat{x}_k^i$.

C. Computational Aspects

The optimization problem (P1) exhibits a high complexity, while the RL-based controller is able to solve the same problem at a low complexity. More specifically, the run-time complexity of (P1) is $O((|\mathcal{U}||\mathcal{L}|)^{|\mathcal{S}|})$, i.e. it increases

exponentially in terms of the number of agents, since it needs to calculate the target received power of all possible combinations of actions to determine the action combination that corresponds to maximum target received power. In RL, the training phase typically consists of extensive simulations that require excessive space and computational complexity depending on the size of the state and action space, but it is performed offline. The prediction/inference stage only requires a lookup index of the Q-table which was learnt during training. In our case, since each agent has its own Q-table, the run-time complexity is $O(|\mathcal{U}||\mathcal{L}|)$, i.e. it is linear with respect to the number of the actions. Information exchanged between the agents is only limited and it is, therefore, not anticipated to affect a potential real-time deployment. The overall steps taken during execution are described in Alg. 2.

Algorithm 2 Execution Steps

Input: $p^{ji}(x_{k-1}|\Upsilon_{1:k-1})$, $\forall j, i$, Q^j , $\forall j$
1: Compute $p^{ji}(x_k|\Upsilon_{1:k-1})$, $\forall j, i$ (Eq. (13)).
2: Compute $\tilde{x}_k^{ji}$, $\forall j, i$ (Eq. (15)).
3: Observe next state s_k^j , $\forall j$.
4: Choose an action $a_k^j = \text{argmax}_{a_k \in \mathcal{A}} Q^j(s_k^j, a_k)$, $\forall j$.
5: Take action a_k^j , $\forall j$.
6: Receive measurement set Υ_k^{ji} , $\forall j, i$.
7: Compute $p^{ji}(x_k|\Upsilon_{1:k})$, $\forall j, i$ (Eq. (14)).
8: Estimate $\hat{x}_k^{ji}$, $\forall j, i$ (Eq. (18)).
9: Exchange $\hat{x}_k^{ji}$, $\forall j, i$ and covariances and fuse to compute $\hat{x}_k^i$, $\forall i$.

V. PERFORMANCE EVALUATION

A. Simulation Setup

To evaluate the performance of the proposed approach, simulation experiments were conducted on a CentOS Linux 7 (Core) server with 10 CPU cores, 43.95GB memory and MATLAB-2020b using the following setup: (i) the rogue drones and UAV agents maneuver in an area of dimensions 100m×100m with randomly initialized state vectors; (ii) for clarity in the simulation results, we assumed that there is no process noise in the actual target trajectory; (iii) the learning rate and exploration parameters linearly decay with each episode iteration, n , and are set as $\alpha = 0.2 - 0.15 \frac{n}{N}$ and $\epsilon = \max\{0, 0.3 * (1 - \frac{n}{0.9N})\}$, respectively; (iv) each agent has its own tile coding function approximator for representing the state space that consists of 2^6 grid tilings. The tile width in m^{jl} and n^{jl} dimensions is set to 1, while in p^{jl} and q^{jl} dimensions is set to 50m; (v) the optimization problem (P1) was simulated running the Gurobi solver; and (vi) the Bayes recursion in Eqs. (13) and (14) is implemented using particle filtering techniques [32]. To facilitate reproducibility of the proposed framework, Table I presents a complete list of simulation parameters.

B. Simulation Results

The first experiment (Experiment 1) considers a single target and two cooperative agents. First, a simulated scenario

TABLE I: Simulation parameters.

Parameter	Value	Parameter	Value
ΔT	1 sec	η	0.003
σ_x	4 m/s ²	R_0	10 m
σ_y	4 m/s ²	Δ_R	[0, 1, 3] m
σ_ω	$\pi/180$ rad/s	N_θ	4
ϕ_0	$\pi/180$ rad	$\mathbb{L}$	{off, -3, 0} dBm
β_ϕ	10^{-5} rad/m	f	2 GHz
ζ_0	1.5 m	Δ	-65 dBm
β_ζ	3×10^{-4} m ⁻¹	K	100
λ	10	N	100000
$p_D^{\max}$	0.99	γ	0.9

that takes 80 time-steps is presented, where the initial target state vector is $[60, 0.9, 30, 0.2, 0.03]^\top$ in units m, m/s², m, m/s², and rad/s, respectively, and the initial positions of agents 1 and 2 are $[65, 39]^\top$ m, and $[59, 30]^\top$ m, respectively. Figure 1 shows the described scenario simulated for both the proposed RL-based approach and the optimization problem (P1) detailed in Section III-B. The target and agent trajectories are shown in Figs. 1a and 1f, where agent 2 tracks in close proximity the target while agent 1 keeps a distance from agent 2 to ensure that the agent-received power is below interference levels, as shown in Figs. 1d and 1i. More specifically, in (P1), agent 1 maneuvers as close to the target, and therefore to agent 2, as possible, which results in the agents receiving interference power very close to the threshold. On the contrary, in the RL-based approach, agent 1 keeps a distance from the target in order to allow for agent 2 to transmit higher levels of jamming power. The tracking error, defined as the Euclidean distance between the ground truth and the estimated target position, is depicted in Figs. 1b and 1g, where the error for the final target state estimate is drawn with a black line. For each time-step in (P1), both agents select to transmit the non-maximum power levels of -3dBm (Fig. 1h), while in the RL-based approach, agent 2 transmits the maximum power level of 0dBm and agent 1 chooses various power levels over time (Fig. 1c). Both approaches result in similar target-received jamming power that is above -50dBm for almost every time-step, as observed from Figs. 1e and 1j. For some time instances, the RL-based approach achieves higher levels of jamming power received at the target compared to (P1), since the agent that closely tracks the target transmits higher power levels.

The second experiment (Experiment 2) consists of two targets and three cooperative agents. A simulated scenario over 80 time-steps is illustrated in Fig. 2 for both the proposed RL-based approach and the optimization problem (P1). In this scenario, agents 1, 2, and 3 are initially positioned at $[51, 83]^\top$ m, $[57, 84]^\top$ m, and $[60, 28]^\top$ m, respectively. The initial target state vector for target 1 is $[60, 0.9, 30, 0.2, 0.03]^\top$ while for target 2 is $[50, -0.8, 80, -0.1, 0.04]^\top$ in units m, m/s², m, m/s² and rad/s, respectively. Both approaches result in similar agent trajectories, as shown in Figs. 2a and 2f, where agents 1 and 3 closely track targets 2 and 1, respectively, while agent 2 remains at a distance to minimize the interference with the rest of the pursuing agents. The

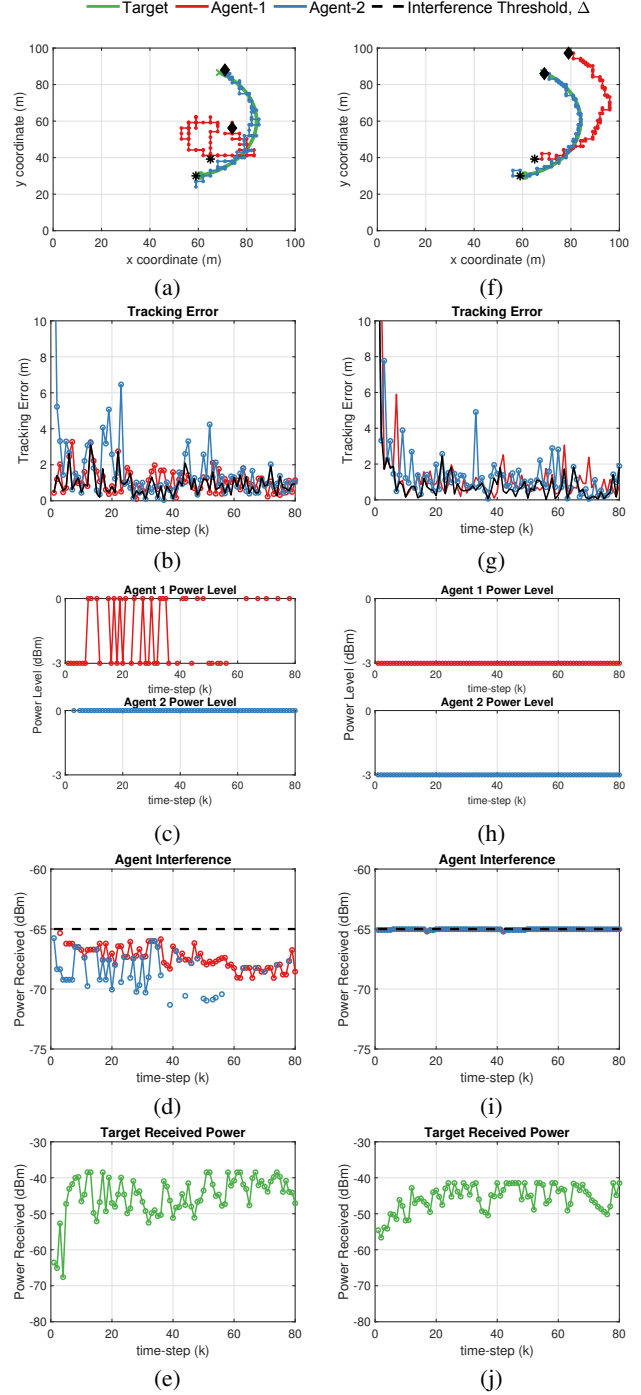


Fig. 1: A 1-target, 2-agent simulated scenario using: (i) RL (a-e), (ii) (P1) (f-j).

tracking error of the final target state estimates, is illustrated in Figs. 2b and 2g. The transmitted power levels of each agent are also shown in Figs. 2c and 2h. The agent-received interference power is always below the predefined threshold (except for a few time instances for the RL-based approach) as depicted in Figs. 2d and 2i. Both targets receive similar levels of jamming power for almost each time-step, as observed from Figs. 2e and 2j, which are significantly higher than the agent received power.

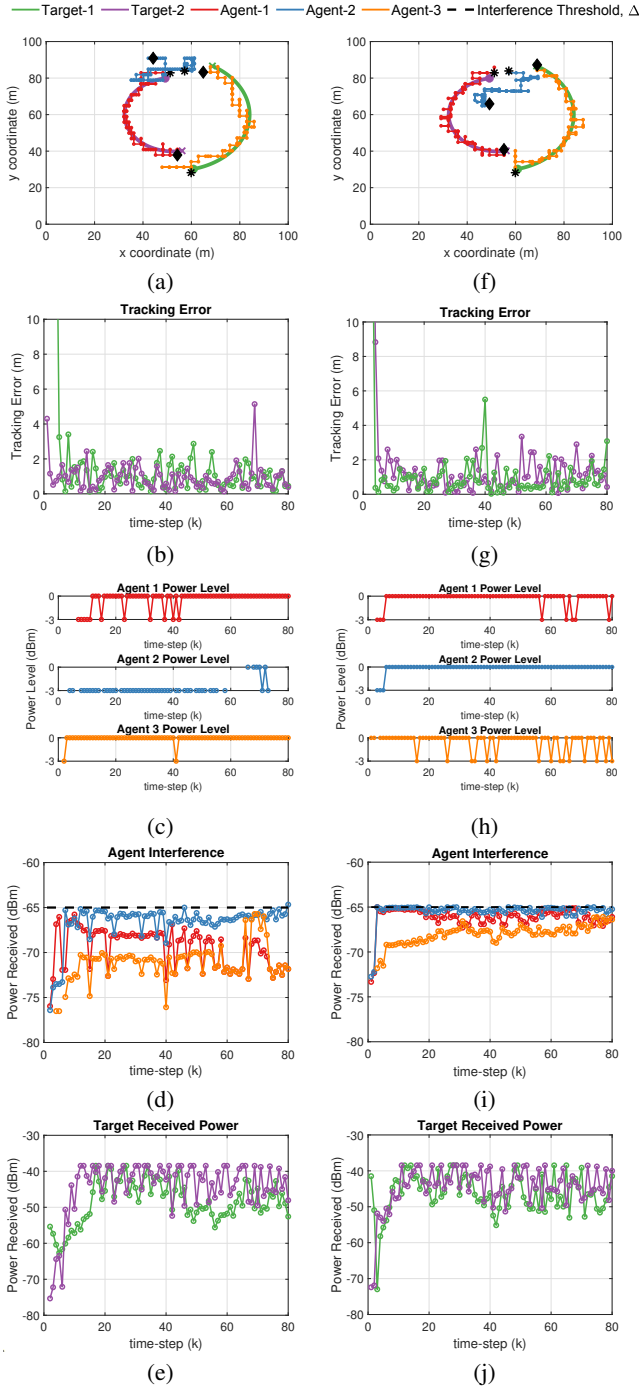


Fig. 2: A 2-target, 3-agent simulated scenario using: (i) RL (a-e), (ii) (P1) (f-j).

Figure 3 illustrates, for both experiments, the average target received power, agent received power, and tracking error per time-step, as the training episodes progress, averaged over 45 independent runs and with a median filter of order 11 applied. As observed, the received power at each target increases with each training episode until it converges to an average of approximately -42dBm and -47dBm for Experiments 1 and 2, respectively, while the received power at each agent remains below interference levels after a few initial episodes. The tracking error slightly reduces with each

training episode and converges to about 1m for Experiment 1 while for Experiment 2 varies between 2 to 8m for each target.

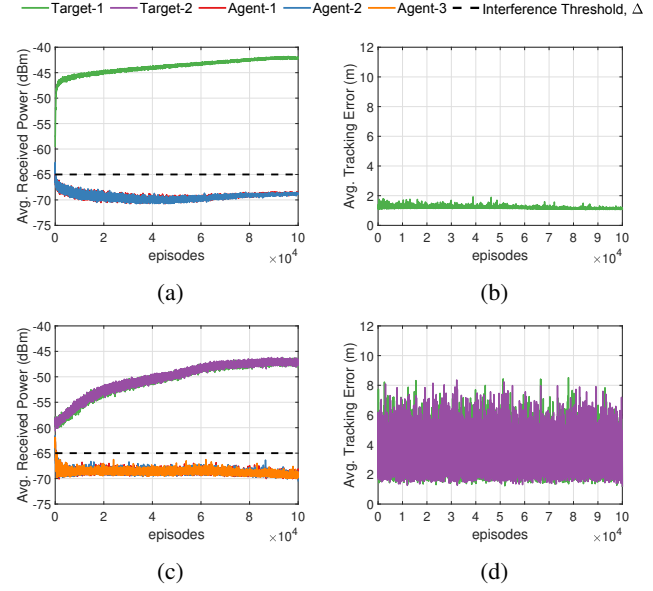


Fig. 3: Training results for the proposed approach: (i) Experiment 1 (a-b), (ii) Experiment 2 (c-d).

To evaluate the performance of the proposed approach, the following metrics are measured by running 1000 testing episodes and averaging over 45 independent runs: the average target received power per time-step, the average agent received power per time-step, and the average tracking error per time-step. For comparison purposes, (P1) was also run for the same 1000 testing episodes. Table II shows the obtained results, demonstrating that the proposed approach performs similar to (P1) for both experiments. Experiment 2 is more challenging than Experiment 1 as indicated by the target received power. The existence of two targets creates scenarios in which their trajectories intersect, and therefore to avoid interference power the agents are forced to either not follow the targets or lower their transmit power levels which result in reducing the target received power.

Further, to verify the speed-up in computation times of the proposed approach (discussed in Section IV-C), the average execution time per time-step (execution steps 1–9 of Alg. 2) is represented in Table III for the same testing episodes. As observed, the RL-based approach requires less time during execution than (P1) for both experiments, while for the second experiment the time is reduced by two orders of magnitude. The reader should note that there could be a further reduction in the computation time, since execution steps 1–4 and 6–8 of Alg. 2 could be processed in parallel for each agent j , whereas in our simulations these steps were processed serially. Overall, the proposed RL-based approach achieves a comparable performance to (P1), and thus is able to effectively solve the cooperative rogue drone(s) jamming problem, while at the same time being more suitable for real-time execution.

TABLE II: Performance evaluation for RL and (P1).

Performance metric		Method			
		RL-based		(P1)	
		Exp. 1	Exp. 2	Exp. 1	Exp. 2
Target received power (dBm)	mean	-42.232	-47.139	-43.797	-44.167
	std	-	-67.151	-	-60.860
Agent received power (dBm)	mean	-68.797	-68.762	-65.087	-66.634
	std	-89.558	-85.678	-Inf	-84.207
Tracking error (m)	mean	1.187	4.104	1.180	3.159
	std	-	0.009	-	0.022

TABLE III: Comparison of execution times for RL and (P1).

Method	Execution Time (s)	
	Experiment 1	Experiment 2
(P1)	0.06	1.13
RL-based	0.02	0.06

VI. CONCLUSIONS

This work investigates the use of multiple UAVs to cooperatively jam multiple rogue drones. A novel scalable RL framework is proposed that selects the mobility and power level control actions for each UAV to best jam the rogue drones. Performance evaluation results indicate that the proposed approach can effectively jam the targets, while avoiding interference amongst the cooperating agents. In fact, the results obtained are comparable to the performance of the optimization problem (P1), while at the same time the execution time is significantly reduced, demonstrating the feasibility of the proposed approach for real-time applications.

Future work will investigate alternative reward functions while now considering evasive targets, and will introduce coordination amongst the agents of the multi-agent RL framework in order to investigate further improvements in performance.

REFERENCES

- [1] Federal Aviation Administration, "FAA aerospace forecast: Fiscal years 2022-2042," 2022. [Online]. Available: https://www.faa.gov/sites/faa.gov/files/2022-06/FY2022_42_FAA_Aerospace_Forecast.pdf
- [2] DRONEII, "Drone market report 2021-2026," 2021. [Online]. Available: <https://droneii.com/the-drone-market-in-2021-and-beyond-5-key-takeaways>
- [3] B. Nassi, A. Shabtai, R. Masuoka, and Y. Elovici, "SoK-security and privacy in the age of drones: Threats, challenges, solution mechanisms, and scientific gaps," *arXiv:1903.05155 [cs.CR]*, 2019.
- [4] G. Lykou, D. Moustakas, and D. Gritzalis, "Defending airports from UAS: A survey on cyber-attacks and counter-drone sensing technologies," *Sensors*, vol. 20, no. 12, 2020.
- [5] K. Wesson and T. Humphreys, "Hacking drones," *Scientific American*, vol. 309, no. 5, pp. 54–59, 2013.
- [6] H. M. Arthur, "Counter-drone systems," *Center for the Study of the Drone, Bard College, Technical Report*, pp. 1–23, 2018.
- [7] S. Park, H. T. Kim, S. Lee, H. Joo, and H. Kim, "Survey on anti-drone systems: Components, designs, and challenges," *IEEE Access*, vol. 9, pp. 42 635–42 659, 2021.
- [8] H. Kang, J. Joung, J. Kim, J. Kang, and Y. S. Cho, "Protect your sky: A survey of counter unmanned aerial vehicle systems," *IEEE Access*, vol. 8, pp. 168 671–168 710, 2020.
- [9] X. Shi, C. Yang, W. Xie, C. Liang, Z. Shi, and J. Chen, "Anti-drone system with multiple surveillance technologies: Architecture, implementation, and challenges," *IEEE Commun. Mag.*, vol. 56, no. 4, pp. 68–74, April 2018.
- [10] S. R. Ganti and Y. Kim, "Implementation of detection and tracking mechanism for small UAS," in *Proc. Inter. Conf. on Unmanned Aircraft Systems (ICUAS)*, 2016, pp. 1254–1260.
- [11] I. Guvenc, F. Koohifar, S. Singh, M. L. Sichitiu, and D. Matolak, "Detection, tracking, and interdiction for amateur drones," *IEEE Commun. Mag.*, vol. 56, no. 4, pp. 75–81, 2018.
- [12] T. Humphreys, "Statement on the security threat posed by unmanned aerial systems and possible countermeasures," *Oversight and Management Efficiency Subcommittee, Homeland Security Committee, US House of Representatives, Washington, DC*, 2015.
- [13] T. Multerer, A. Ganis, U. Prechtel, E. Miralles, A. Meusling, J. Mietzner, M. Vossiek, M. Loghi, and V. Ziegler, "Low-cost jamming system against small drones using a 3D MIMO radar based tracking," in *Proc. European Radar Conf. (EURAD)*, 2017, pp. 299–302.
- [14] A. H. Abunada, A. Y. Osman, A. Khandakar, M. E. H. Chowdhury, T. Khatatb, and F. Touati, "Design and implementation of a RF based anti-drone system," in *Proc. IEEE Inter. Conf. on Informatics, IoT, and Enabling Technologies (ICIoT)*, 2020, pp. 35–42.
- [15] N. Souli, R. Makrigiorgis, A. Anastasiou, A. Zacharia, P. Petrides, A. Lazanas, P. Valianti, P. Kolios, and G. Ellinas, "HorizonBlock: Implementation of an autonomous counter-drone system," in *Proc. Inter. Conf. on Unmanned Aircraft Systems (ICUAS)*, 2020, pp. 398–404.
- [16] V. U. Castrillo, A. Manco, D. Pascarella, and G. Gigante, "A review of counter-uas technologies for cooperative defensive teams of drones," *Drones*, vol. 6, no. 3, 2022.
- [17] A. Guerra, D. Dardari, and P. M. Djuric, "Dynamic radar networks of UAVs: A tutorial overview and tracking performance comparison with terrestrial radar networks," *IEEE Vehicular Technology Magazine*, vol. 15, no. 2, pp. 113–120, 2020.
- [18] J. Rothe, M. Strohmeier, and S. Montenegro, "A concept for catching drones with a net carried by cooperative UAVs," in *Proc. IEEE Inter. Symp. on Safety, Security, and Rescue Robotics*, 2019, pp. 126–132.
- [19] M. R. Brust, G. Danoy, P. Bouvry, D. Gashi, H. Pathak and M. P. Gonçalves, "Defending against intrusion of malicious UAVs with networked UAV defense swarms," in *Proc. IEEE 42nd Conf. on Local Computer Networks Workshops*, 2017, pp. 103–111.
- [20] P. Valianti, S. Papaioannou, P. Kolios, and G. Ellinas, "Multi-agent coordinated close-in jamming for disabling a rogue drone," *IEEE Trans. on Mobile Computing*, vol. 21, no. 10, pp. 3700–3717, 2022.
- [21] —, "Multi-agent coordinated interception of multiple rogue drones," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2020, pp. 1–6.
- [22] Y. Song, M. Steinweg, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing with deep reinforcement learning," in *Proc. IEEE Int. Conf. Intell. Robots Syst. (IROS)*, 2021, pp. 1205–1212.
- [23] X. Wang, M. Cen Gursoy, T. Erpek, and Y. E. Sagduyu, "Jamming-resilient path planning for multiple UAVs via deep reinforcement learning," in *Proc. IEEE Inter. Conf. on Commun. Workshops (ICC Workshops)*, 2021, pp. 1–6.
- [24] M. Theile, H. Bayerlein, R. Nai, D. Gesbert, and M. Caccamo, "UAV coverage path planning under varying power constraints using deep reinforcement learning," in *Proc. IEEE Int. Conf. Intell. Robots Syst. (IROS)*, 2020, pp. 1444–1449.
- [25] L. S. B. Vlahov, E. Squires and C. Pippin, "On developing a UAV pursuit-evasion policy using reinforcement learning," in *Proc. 17th IEEE Inter. Conf. on Mach. Learn. and Appl. (ICMLA)*, 2019, pp. 859–864.
- [26] M. Min, L. Xiao, D. Xu, L. Huang, and M. Peng, "Learning-based defense against malicious unmanned aerial vehicles," in *Proc. IEEE 87th Vehicular Technology Conf. (VTC)*, 2018, pp. 1–5.
- [27] R. S. Sutton and A. G. Barto, *Introduction to Reinforcement Learning*. MIT Press Cambridge, 1998.
- [28] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation: Theory Algorithms and Software*. John Wiley & Sons, 2004.
- [29] M. Roth, G. Hendeb, and F. Gustafsson, "EKF/UKF maneuvering target tracking using coordinated turn models with polar/Cartesian velocity," in *Proc. 17th Inter. Conf. on Inform. Fusion*, 2014, pp. 1–8.
- [30] Z. Chen, "Bayesian filtering: From Kalman filters to particle filters, and beyond," Adaptive Syst. Lab., McMaster University, Hamilton, ON, Canada, Technical Report, 2003.
- [31] P.O. Arambel, C. Rago, and R.K. Mehra, "Covariance intersection algorithm for distributed spacecraft state estimation," in *Proc. IEEE American Control Conf. (ACC)*, 2001, pp. 4398–4403.
- [32] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, "A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking," *IEEE Trans. Signal Process.*, vol. 50, no. 2, pp. 174–188, 2002.